



UNIVERSITÀ
DEGLI STUDI
DI BERGAMO

Dipartimento
di Ingegneria Gestionale,
dell'Informazione e della Produzione

Strategie di ricerca informata o eurostica

Intelligenza Artificiale

Ingegneria Informatica

Docente

Domenico Fabio Savo

Ricerca non informata vs ricerca informata

Le **strategia di ricerca non informata** trovano soluzioni (spesso non efficienti) a problemi generando **sistematicamente** nuovi stati che sono poi verificati rispetto all'obiettivo.

Sono anche dette **cieche** perché non considerano l'obiettivo, cioè non usano nessuna informazione sull'obiettivo a cui stanno mirando finché non ci capitano su.

Le **strategie di ricerca informata** definiscono invece metodologie risolutive che usano informazione **euristica** per scegliere ad ogni passo quali sono i nodi più convenienti da esplorare.

Utilizzando **conoscenza aggiuntiva (l'euristica)** specifica al problema e al dominio di applicabilità, trovano soluzioni (*a volte non ottime*) in modo più efficiente, cioè riducono il costo della procedura di ricerca.



Euristica – Ricerca euristica

Euristica (definizione di Feigenbaum e Feldman, 1963)

Una euristica (regola o metodo) è una regola, strategia, semplificazione o qualunque altro strumento che drasticamente limita la ricerca di soluzioni nello spazio degli stati del problema.

Le euristiche non garantiscono soluzioni ottimali; non garantiscono soluzioni in ogni caso.

Una euristica utile è ciò che offre soluzioni che sono abbastanza buone il più delle volte.



Tecniche euristiche

Euristica è una qualunque **tecnica che migliora le prestazioni del compito di risoluzione** del problema nel caso **medio**, ma non le migliora necessariamente nel caso peggiore.

- Negli **algoritmi di ricerca** si riferisce ad una funzione (h) che fornisce una stima del costo della soluzione.
- Nei **sistemi esperti** le tecniche euristiche erano viste come “regole empiriche” che gli esperti del dominio potevano usare per generare “buone soluzioni” senza ricercare esaustivamente.

Se le euristiche vengono espresse come regole, si parla di **sistemi basati su regole**.



Strategia di ricerca informata

Le tecniche euristiche **non sono facilmente adattabili** a nuovi problemi.

Esiste l'interesse nello sviluppare algoritmi di ricerca euristici generalizzati le cui proprietà possano essere studiate indipendentemente dal particolare dominio e dal programma che li usi.

*Dato un problema generale di rappresentazione, le tecniche di ricerca euristica più basilari **possono essere studiate come variazione dei metodi di ricerca cieca** per lo stesso tipo di rappresentazione del problema.*



Come usare una euristica

I punti in cui applicare l'informazione euristica sono relativi a:

- **decidere quali nodi espandere** (invece di applicare tecniche in profondità od in ampiezza) nel corso dell'espansione del nodo,
- **decidere quale successore/i generare** (invece di una generazione cieca di tutti i possibili successori ad un dato istante)
- **decidere che certi nodi devono essere scartati**, od eliminati, dall'albero di ricerca.



Strategia di ricerca

L'idea generale è quella di ***espandere il nodo che sembra “più promettente”***.

In genere la specifica strategia di ricerca si distingue per la diversa **funzione di valutazione** usata per determinare il prossimo nodo da espandere.

La **funzione di valutazione** determina il valore di desiderabilità (o non desiderabilità) dell'espansione di ogni specifico nodo.



Funzione di valutazione (euristica)

In una strategia di **ricerca informata** la funzione di valutazione sfrutta conoscenza specifica del dominio applicativo per fornire **suggerimenti** su dove si potrebbe trovare l'obiettivo.

I *suggerimenti* hanno la forma di una **funzione euristica** denotata con **$h(n)$** :

$h(n)$ = costo stimato del cammino meno costoso dallo stato del nodo n a uno stato obiettivo.

Ogni funzione euristica è **specifica al problema**, ovvero viene definita a seconda del problema che si sta risolvendo

Ad esempio, in problemi di ricerca di un itinerario possiamo stimare la distanza dallo stato corrente a uno stato obiettivo calcolando la distanza in linea d'aria tra due punti sulla mappa.



Ricerca best-first greedy o ricerca “golosa”

La ricerca **best-first greedy** è una forma di ricerca **best-first** che espande prima il nodo con il **valore più basso di $h(n)$** , cioè quello che *appare* più vicino all’obiettivo.

Si considera quindi una funzione di valutazione **$f(n) = h(n)$** .

L’algoritmo è detto “**goloso**” perché ad ogni passo cerca sempre di arrivare il più possibile vicino a un obiettivo secondo le sue percezioni date da **$h(n)$** .

Tale strategia può però portare a **risultati peggiori** di quelli ottenibili procedendo con maggiore cautela.



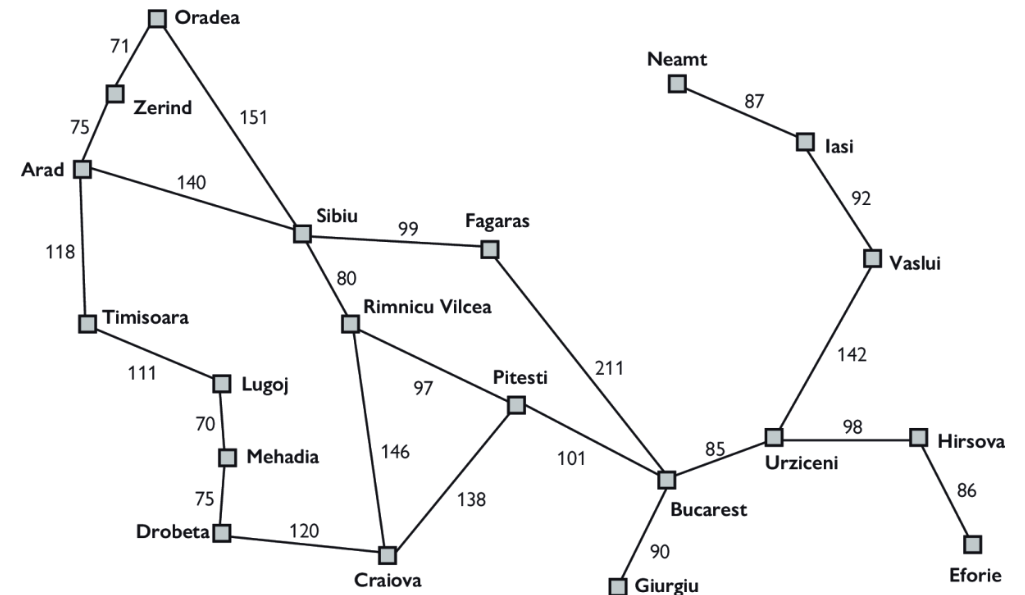
Ricerca best-first greedy: esempio

Vediamo come funziona con il nostro problema dell'itinerario in Romania usando l'euristica della **distanza in linea d'aria**, che indicheremo con h_{DLA}

h_{DLA} :

Arad	366	Mehadia	241
Bucarest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Mapa con distanze reali:

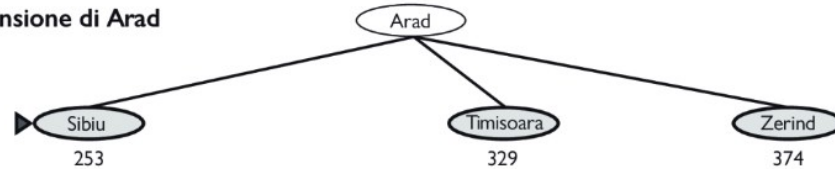


Ricerca best-first greedy: esempio

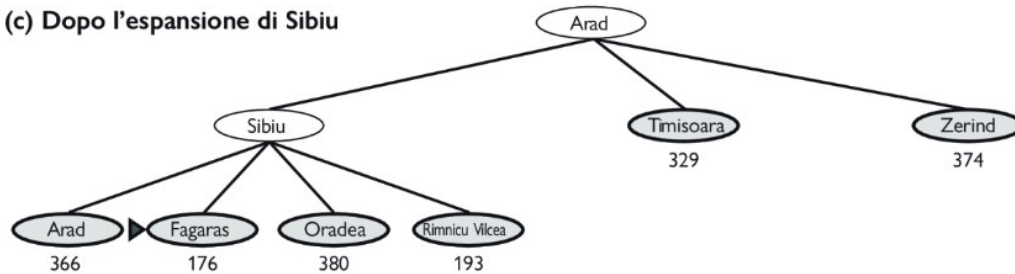
(a) Stato iniziale



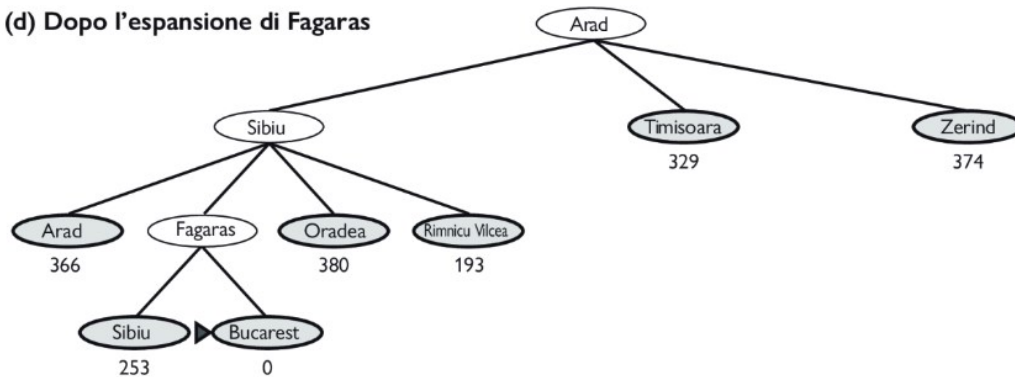
(b) Dopo l'espansione di Arad



(c) Dopo l'espansione di Sibiu



(d) Dopo l'espansione di Fagaras



Il percorso scelto dall'algoritmo sarà:
Arad – **Sibiu** – **Fagaras** – **Bucarest**

Con un costo (reale) di **450**

Il percorso (ottimo) che invece
passa per Rimnicu Vilcea e Pitesti
è invece di **418**

Ricerca best-first greedy: Algoritmo

Come la ricerca a costo uniforme, l'algoritmo può essere implementato come chiamata di **Ricerca-Best-First** con la funzione euristica $h(n)$ come **funzione di valutazione**

```
function Ricerca-Best-First-Greedy(problema) returns un nodo soluzione o fallimento
return Ricerca-Best-First(problema,  $h(n)$ )
```

In questo modo, i nodi della *frontiera* saranno ordinati secondo la funzione euristica $h(n)$. Come conseguenza i nodi che, in base ad $h(n)$, sembrano **più promettenti** saranno espansi per primi.



Ricerca best-first greedy: osservazioni e complessità

La ricerca best-first greedy si comporta come una ricerca in profondità: segue un cammino fino all'obiettivo, per tornare indietro quando si imbatte in un vicolo cieco.

Non è ottimale e può essere incompleta: stessi difetti della ricerca in profondità, si incammina lungo un percorso, e può finire in rami infiniti. È invece completa in uno spazio degli stati finito.

Valutazione: ricerca **non completa** ed **non ottimale**

- **Complessità temporale:** nel caso peggiore, se abbiamo profondità d e fattore di ramificazione b il numero massimo di nodi espansi sarà $O(b^d)$
- **Complessità spaziale:** $O(b^d)$, coincide con quella temporale perché tiene in memoria tutti i nodi.

Con una buona funzione euristica, la complessità spaziale e temporale possono essere ridotte sostanzialmente



Algoritmo di ricerca A*

Algoritmo di ricerca informata che invece di considerare solo la distanza dall'obiettivo, considera anche il "costo" nel raggiungere il nodo **n** dalla radice.

È quindi una ricerca **best-first** che utilizza la funzione di valutazione:

$$f(n) = g(n) + h(n)$$

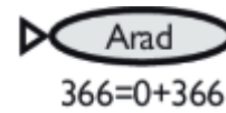
dove **g(n)** è il costo del cammino dal nodo **iniziale** al nodo **n** e **h(n)** rappresenta il costo **stimato** (funzione euristica) del cammino più breve da **n** a uno stato obiettivo, per cui abbiamo:

f(n) = costo stimato del cammino migliore che continua da n fino a un obiettivo.



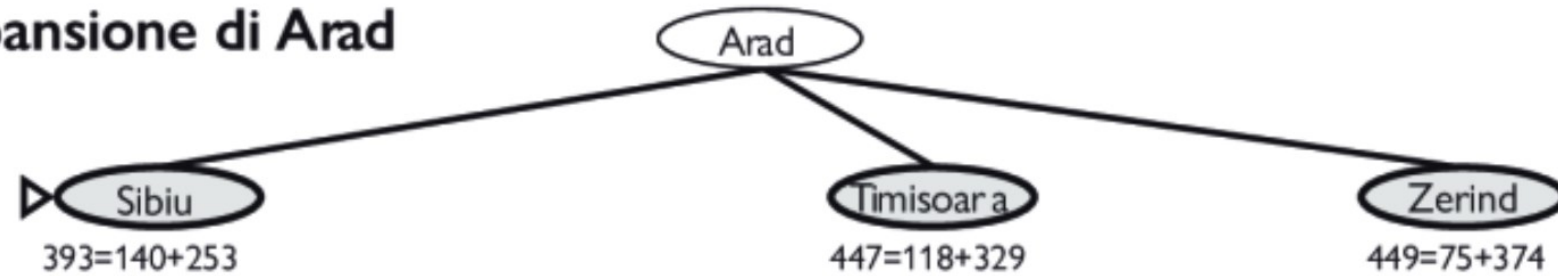
Algoritmo di ricerca A*: esempio

(a) Stato iniziale



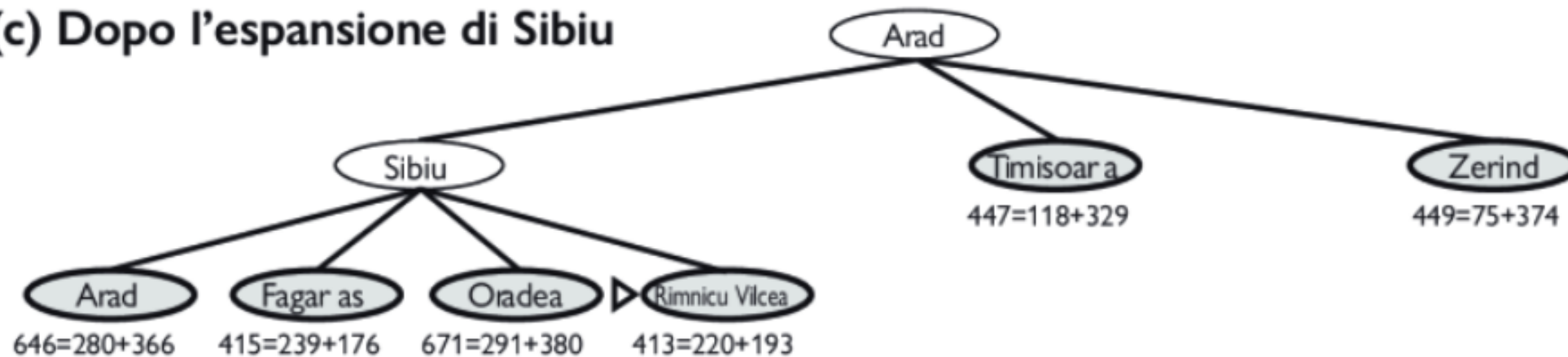
Algoritmo di ricerca A*: esempio

(b) Dopo l'espansione di Arad



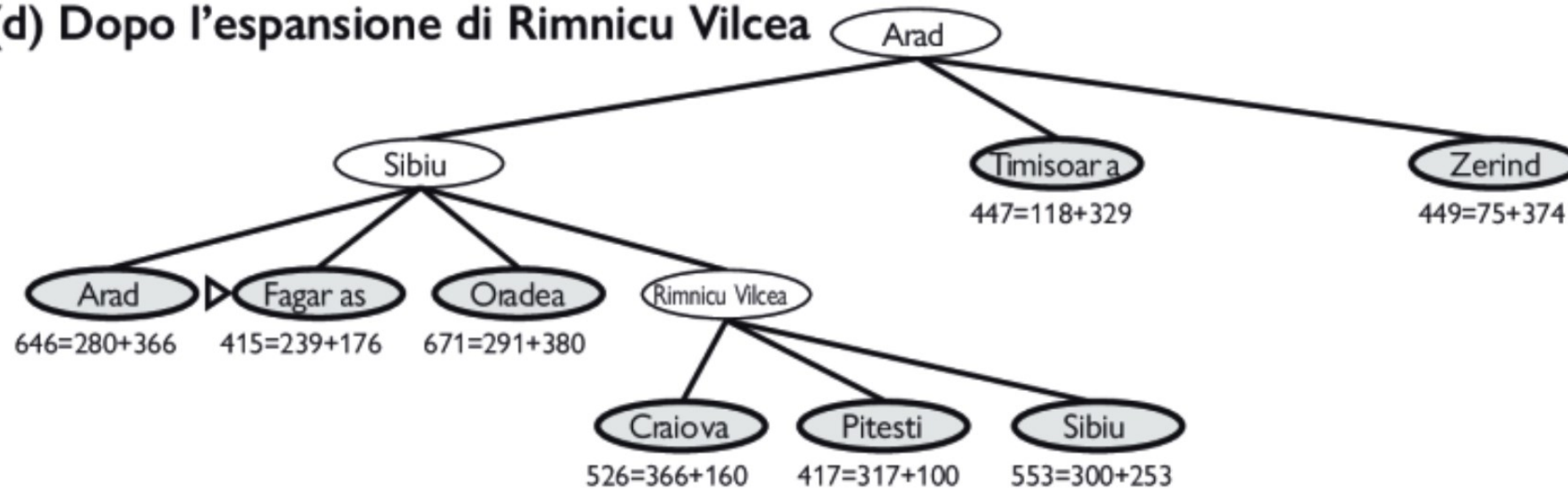
Algoritmo di ricerca A*: esempio

(c) Dopo l'espansione di Sibiu



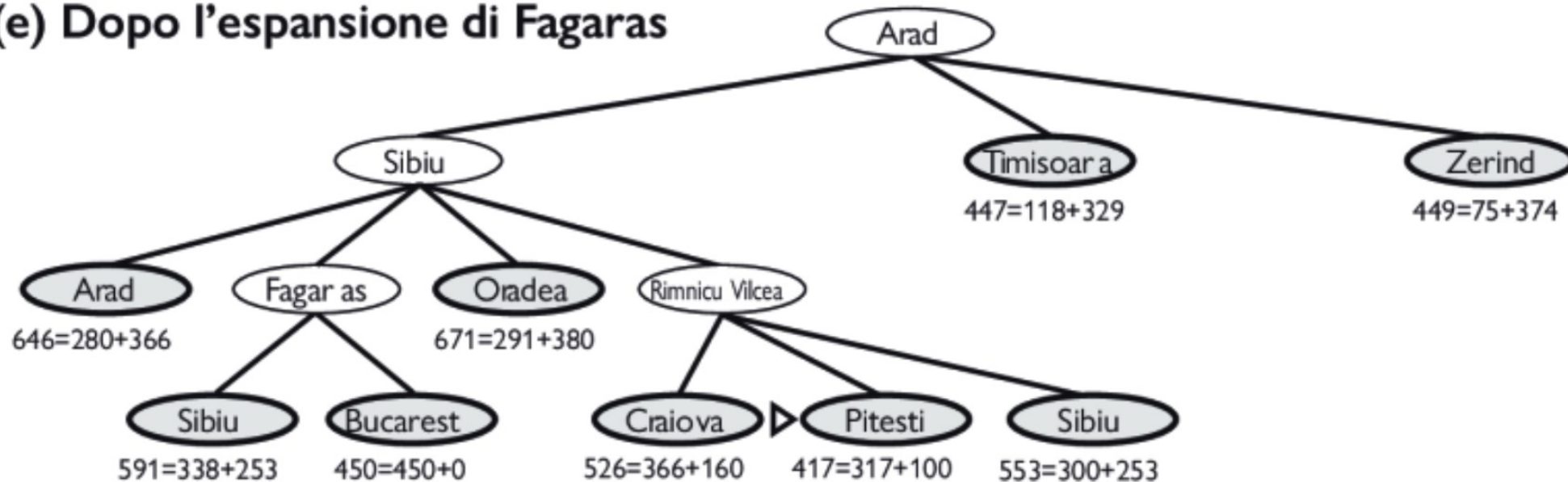
Algoritmo di ricerca A*: esempio

(d) Dopo l'espansione di Rimnicu Vilcea



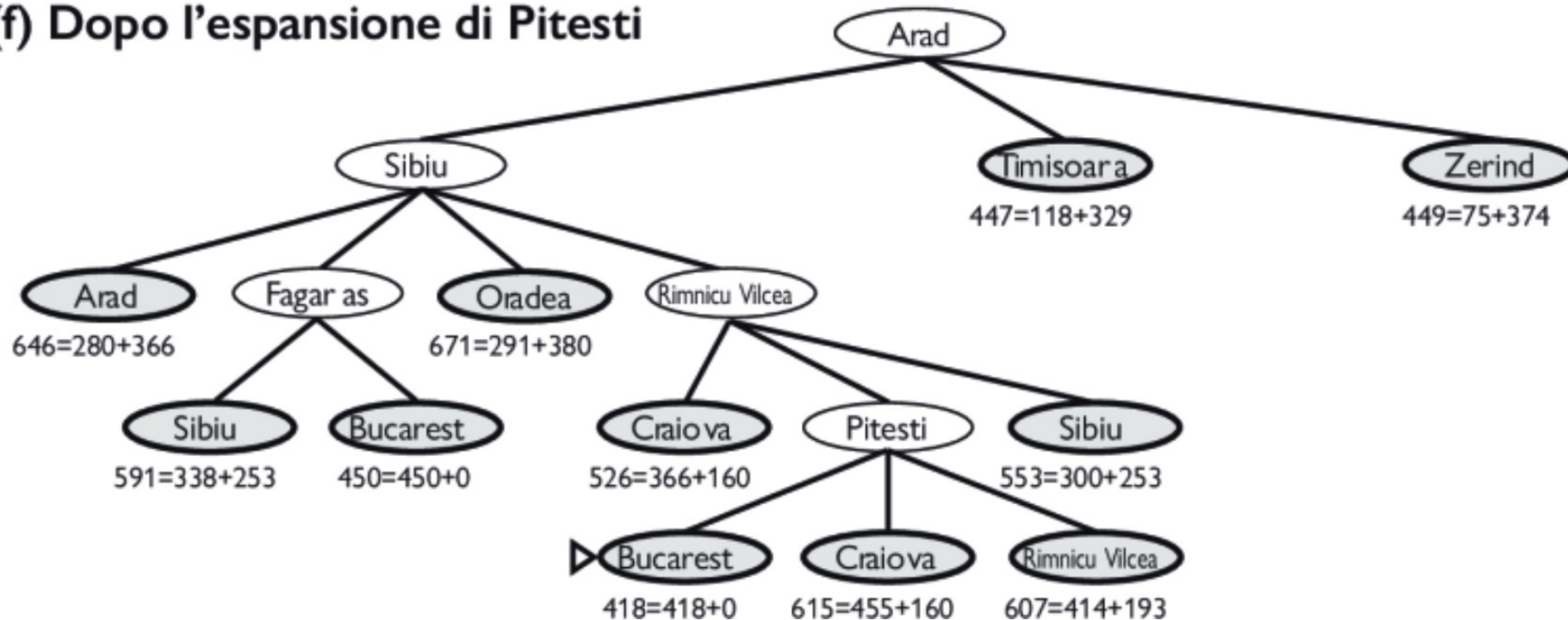
Algoritmo di ricerca A*: esempio

(e) Dopo l'espansione di Fagaras



Algoritmo di ricerca A*: esempio

(f) Dopo l'espansione di Pitesti



Algoritmo di ricerca A*: rompicapo dell'8

2	8	3
1	6	4
7		5

Stato iniziale

1	2	3
8		4
7	6	5

Stato obiettivo

Azione: spostare la casella "vuota"

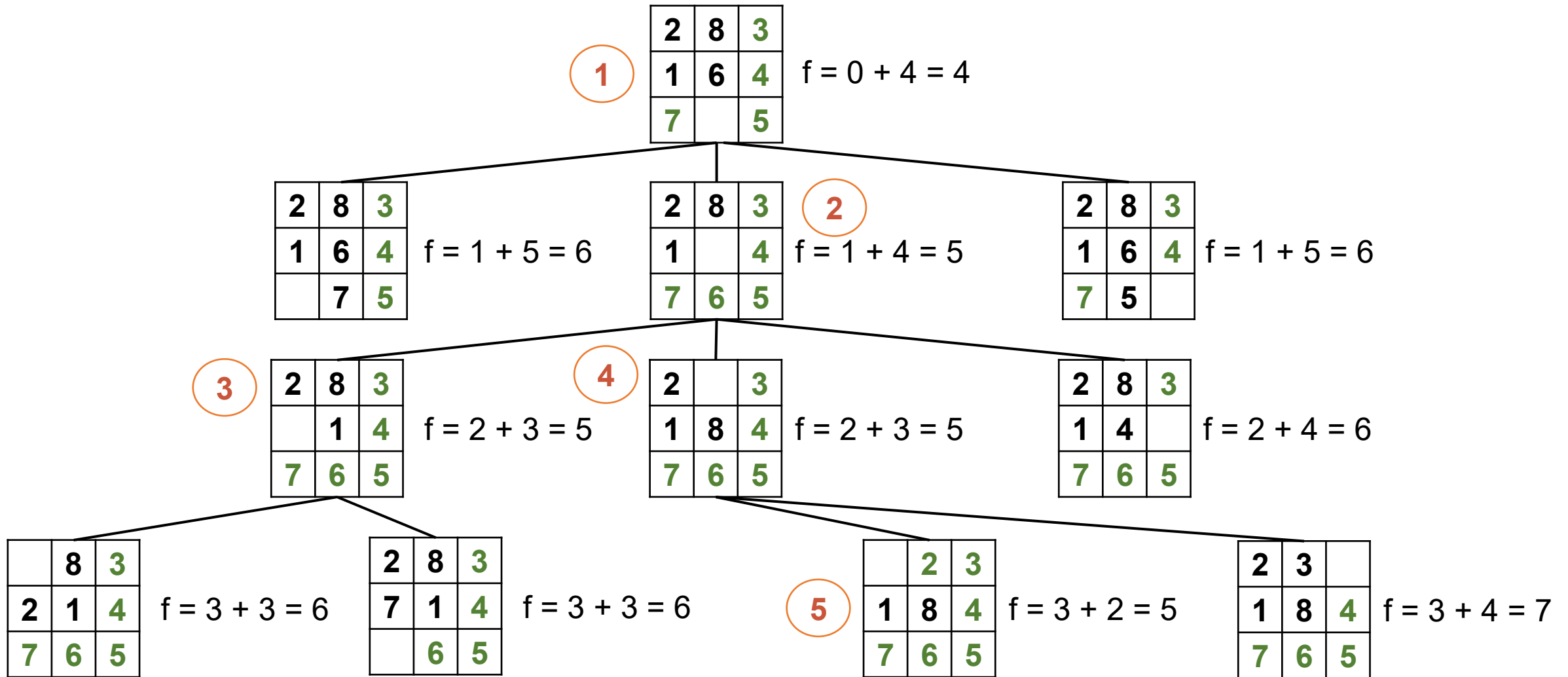
Costo di ogni azione (g): unitario (1)

Costo stimato dallo stato obiettivo (h): *"numero di caselle fuori posto"*

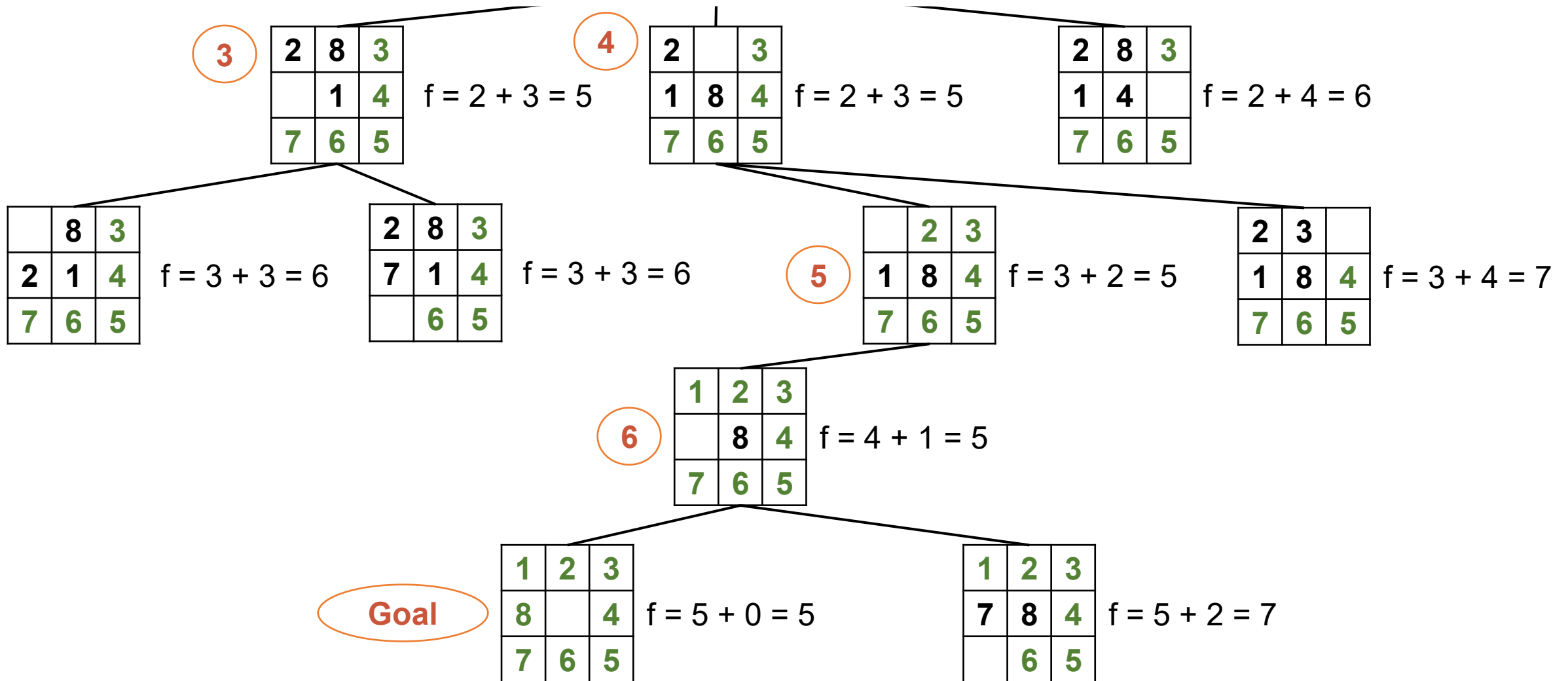
Funzione di valutazione: $f(n) = g(n) + h(n)$



Algoritmo di ricerca A*: rompicapo dell'8



Algoritmo di ricerca A*: rompicapo dell'8



Algoritmo di ricerca A*: euristiche ammissibili

La ricerca A* è **completa** in spazi degli stati finiti.

Il fatto che sia o no ottimale rispetto al costo dipende dall'euristica:

Indichiamo con $h(n)$ la vera distanza fra il nodo corrente n e quello obiettivo.
Diciamo che un funzione euristica $h'(n)$ è **ammissibile** se:

$$\forall n \quad h'(n) \leq h(n) \text{ (funzione ottimistica)}$$

Teorema: Se $h'(n) \leq h(n)$ per ogni nodo, allora l'algoritmo A* troverà sempre il nodo obiettivo ottimale.

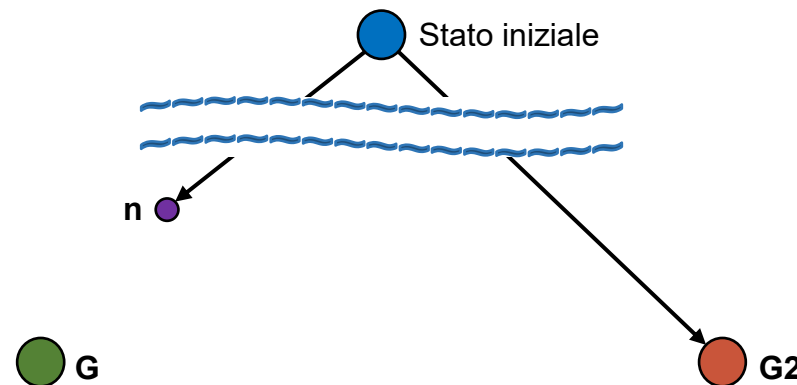
Ovviamente, l'euristica perfetta $h' = h$ è sempre ammissibile così come $h' = 0$



Ottimalità ricerca A* con euristiche ammissibili: dimostrazione (1 di 2)

Teorema: Se $h'(n) \leq h(n)$ per ogni nodo, allora l'algoritmo A* troverà sempre il nodo obiettivo ottimale.

Dimostrazione: Supponiamo, per assurdo, che l'algoritmo abbia raggiunto uno stato obiettivo (goal **G2**) seguendo un percorso sub-ottimo e di averlo inserito nella frontiera. Sia **n** un nodo non espanso nella frontiera tale che **n** è nella strada più breve verso il goal ottimo **G**.



Ottimalità ricerca A* con euristiche ammissibili: dimostrazione (1 di 2)

- $f(G2) = g(G2)$ poiché $h'(G2) = 0$ ($G2$ è una soluzione)
- $g(G2) > g(G)$ poiché $G2$ è sub-ottima (ha costo di cammino maggiore)
- $f(G) = g(G)$ poiché $h'(G) = 0$ (G è una soluzione)

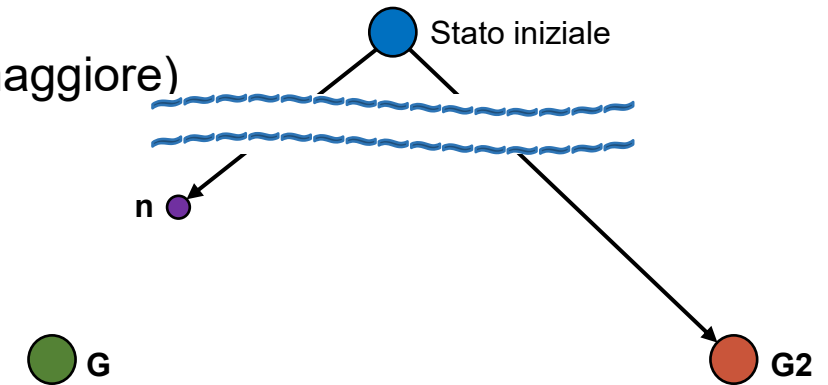
quindi:

- $f(G2) > f(G)$ da sopra
- $h'(n) \leq h(n)$ poiché h' è ammissibile

quindi:

- $g(n) + h'(n) \leq g(n) + h(n)$
- $g(n) + h(n) = f(n)$ per definizione
- $f(n) \leq f(G)$ dato che n è sulla strada che porta a G e quindi:

$f(G2) > f(n)$, e **A* non selezionerà mai G2 per l'espansione** (contraddizione).



Ricerca A*: Algoritmo

L'algoritmo può essere implementato come chiamata di **Ricerca-Best-First** con la funzione euristica $f(n) = g(n) + h(n)$ come **funzione di valutazione**

```
function Ricerca-Best-First-Greedy(problema) returns un nodo soluzione o fallimento  
    return Ricerca-Best-First(problema,  $f(n)$ )
```

Esistono però anche altre implementazioni più efficienti



Ricerca A*: osservazioni e complessità

La ricerca A* è **ottimale** e **completa** (se non esistano infiniti nodi con n tale per cui $f(n) < g(G)$)

Valutazione: ricerca **completa** ed **ottimale**

- **Complessità temporale:** nel caso peggiore, se abbiamo profondità d e fattore di ramificazione b il numero massimo di nodi espansi sarà $O(b^d)$
- **Complessità spaziale:** $O(b^d)$, coincide con quella temporale perché tiene in memoria tutti i nodi.

Rinunciando all'ottimalità è possibile abbassare la complessità.

